

Java O-O Program Visualizer/Animator
Milestone 1

By Prottasha Deb, Angelina Martis,
Arya Patel & Aiden Stensrud

Our Program



01

User Input

Users can input a Java program which is parsed into visualizer instructions.



02

Visualize

The visualizer will display the input program's memory data



03

Animate

Users can iterate through the program and see the memory diagram shift

Functional/Non-Functional Requirements

Functional

- Enter and Load Source Files
  - App shall allow user to input, edit, and clear source code in the editor
- Analyze Supported Source
  - App shall analyze input within documented Java subset and provide representation to generate exec. states
- Generate Execution States
  - App shall generate an ordered trace of the program at each execution step
- Navigate Forward
  - App shall provide toolbar controls to navigate through execution states
- Synchronize Code and Diagram
  - App shall update diagram according to the active step
- Display the Memory Model
  - App shall display memory state using different shapes and symbols to distinguish active frames and objects

Non-Functional

- Usability
  - App GUI must be easy to access for users and intuitive for those with limited technical expertise
- Maintainability/Testable
  - App must maintain a reliable code structure that can be easy to modify and restructure
- Portability/Compatibility
  - App must be able run across different OPs, devices, etc.

Test Plans and Methodology

Core Purpose: Ensure Java programs generate correct memory diagrams and sync with execution controls.

Execution Environment: Standardized across Windows 11 and macOS using independent expected states to prevent bias.

Parser & Memory State Testing
Validates primitives, object allocations, and references
Strictly tests handling of invalid syntax and documented unsupported constructs

Integration, Animation, & UI
Tests full workflows, step-throughs, and animation steps
Tests visual layouts (labels, arrows) and interface/speed controls

Risks & Project Management

Risk Management: High-severity layout overlaps are mitigated by progressive testing, auto-spacing checks, and regression runs.

Phase 1 Deliverables: Finalized test plan, reusable Java samples, and checked expected states

Team & Communication: Defines task ownership and coordinates team workflows using GitHub, Google Docs, Meets, and email.

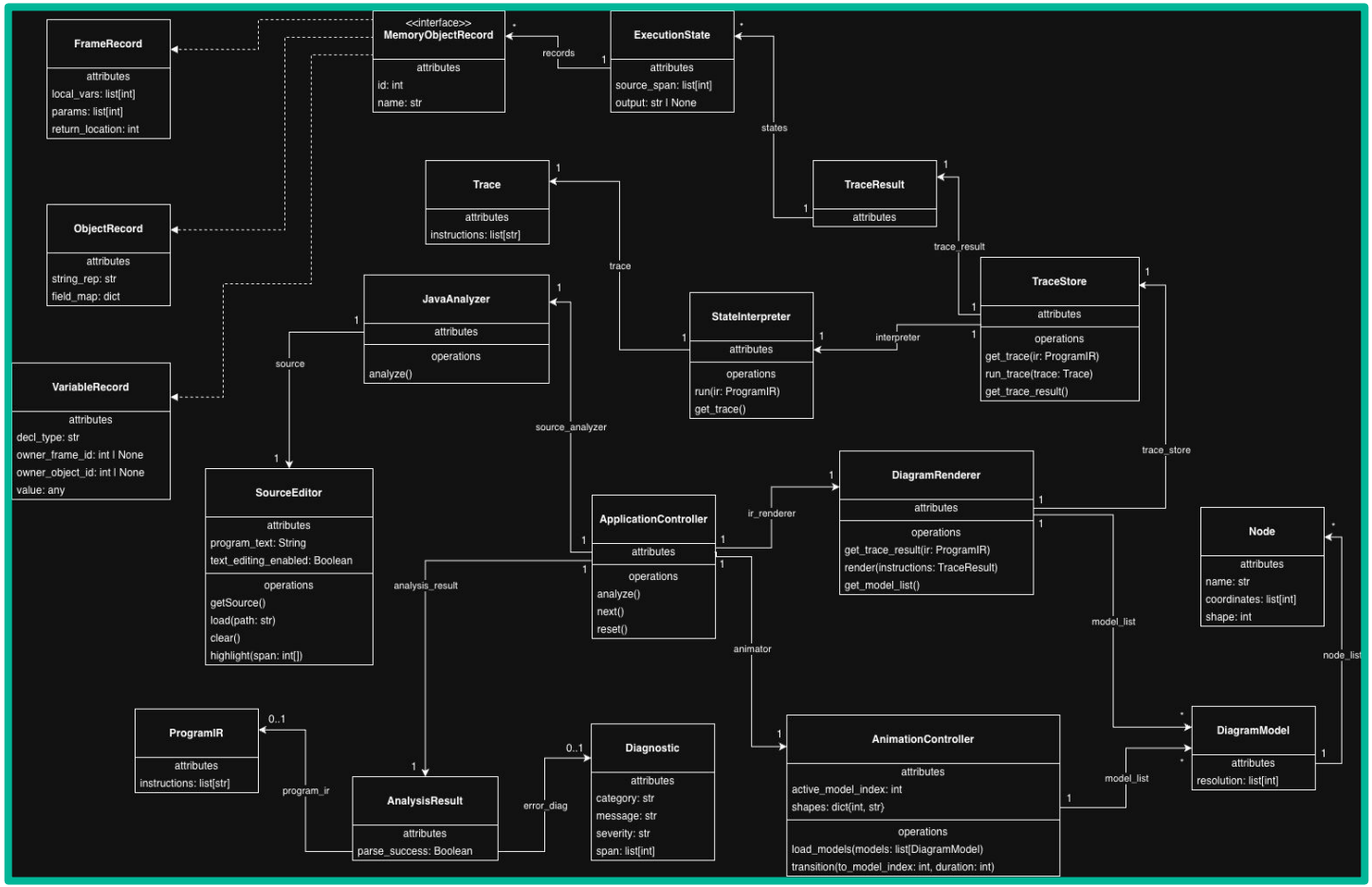
Milestone 1 Focus	• **Test Planning:** Draft test plan and document language subset gaps • **Diagram Verification:** Create Java input samples and expected memory diagrams • **Early Reviews:** Inspect parser prototypes and interface mockups

Design Documentation

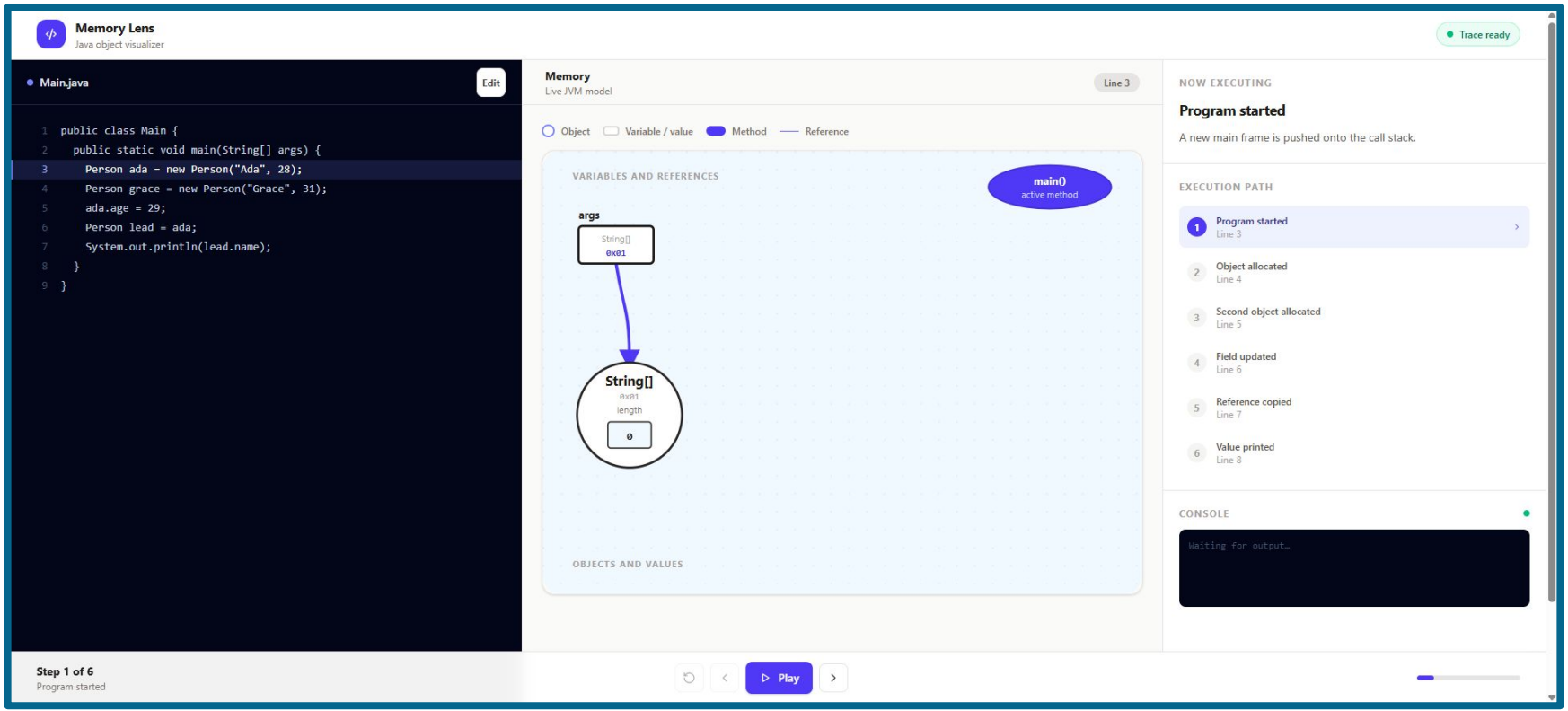
System Overview: Split into four different runtime workflows: parsing code, checking result, converting to instructions, and displaying.

Data & Memory Model:
Tracks references via unique ObjectId values, ensuring shared references point to the same object without duplication

Class Diagram



User Interface & Interactive Controls



Java O-O Visualizer Interface Demo

```
public class Car {
   int speed;
   double direction;
   string type;

   public void accelerate (int increase) {
      this.speed += increase;
   }
   public static void main ...() {
      Car c1 = new Car();
      Car c2 = new Car(20, 90, "sedan");
      c1.accelerate(50);
      Car c3 = c2;
      int hello = 3;
      String toPrintOut = "Hello";
      System.out.println(toPrintOut);
   }
```

Output

Technical Challenges

RESOLVED - Diagram Implementation

Team is very familiar with client's intended design after a demo

Dynamic Organization

Difficult to automatically organize diagrams so information remains easy to understand



RESOLVED - Compiler Knowledge

Team is now familiar with common compiler strategies

Complex Programs

Large programs can create crowded memory diagrams

Animation Complexity

Accurately animate low-level program information for complex Java programs

Milestone 1 Task Matrix Completion

Task	Completion	Arya	Aiden	Prottasha	Angelina
Resolve technical challenges	100%	-	lack of compiler knowledge	-	-
Design memory diagram demo	100%	design 50%	give input	give input	design 50%
Compare and select the application type	100%	web	-	-	desktop
Compare and select Collaboration Tools	100%	presentation	programs	documents	communication, task calendar
Compare and select technical tools/strategies	100%	-	java code parser	memory instruction animator	-
Requirement Document	100%	write 10%	write 30%	write 50%	write 10%
Design Document	100%	write 10%	write 50%	write 30%	write 10%
Test Plan	100%	write 10%	write 30%	write 50%	write 10%

Milestone 2 Task Matrix

Task	Arya	Aiden	Protasha	Angelina
Implement, test, and demo Java code parser	-	100%	-	-
Implement, test, and demo simple memory diagram generator	20%	-	-	80%
Conceptualize, draft, and demo memory diagram animations	5%	5%	85%	5%
Resolve technical challenge	Memory diagram design for complicated memory states	-	-	-